

model validation

typical supervised learning scheme

- 1 selection of the class of model $m(\cdot)$ (e.g. polynomial regression)
- 2 hyperparameters selection (e.g. polynomial degree)
- 3 training the model on TR
- 4 use of the trained model to make prediction on unseen new data

→ NOT USED
DURING TRAINING

steps (1) and (2) crucial for the prediction performance of the model

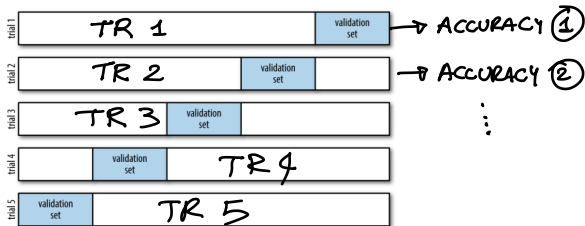
- how to select the model?
- how to select hyperparameters?

a model **validation** is needed!

model validation

- **holdout set**: remove a subset of available data to create a testing set (not used in the training!)
if testing set small scarce significance, if too much data subtracted to the training → IMPORTANT REPRESENTATIVE OCCURRENCES NOT USED DURING TRAINING
↳ USED TO ASSESS GENERALIZATION
- **k-fold cross-validation**: train the model k times (trials or folds), at each trial the dataset is divided in a training and a **validation set** (used to measure the accuracy), the validation set is a $\frac{1}{k}$ fraction of the number of available data, the final performance is the average over all k folds

COMMON CHOICES:
 $k=5, k=10$



- ▶ all data considered one time in the validation
- ▶ time consuming

↳ TRAIN THE MODEL K TIMES

FINAL ACCURACY
IS AVERAGED
OVER THE K TRIALS



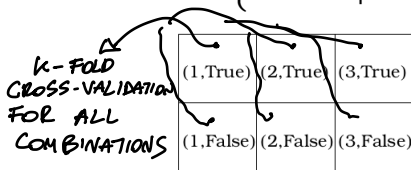
which model/hyperparameters to select?

optimization of the model/hyperparameters is too expensive!

model/hyperparameters selection done heuristically via **grid-search**

- 1 for every hyperparameter (including model structure) choose a set of reasonable values (all possibly) $\rightarrow$ **GRID**
- 2 perform a k-fold cross-validation for every combination of hyperparameters values
- 3 select the hyperparameters values in which the best model accuracy is achieved

example: polynomial regression $\Rightarrow$ $\begin{cases} q : & \{1, 2, 3\} \\ \text{fit_intercept} : & \{True, False\} \end{cases}$



• IS NOT EASY TO DETERMINE GOOD GRID VALUES
very time consuming!

`sklearn.model_selection.GridSearchCV()`

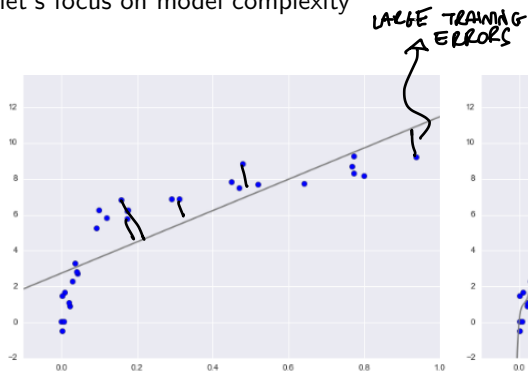
improve model performance

what if the trained model does not work well?

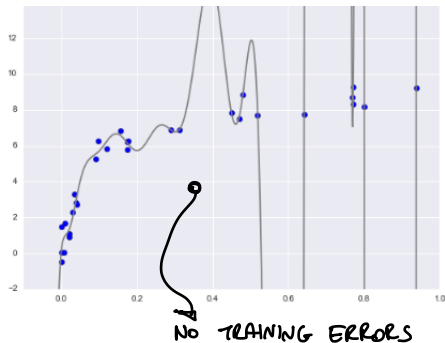
- adopt a more complex/expressive model?
- adopt a less complex/expressive model?
- collect more training data?
- increase the number of features?
- decrease the number of features?
- improve the dataset quality?

often in machine learning the answer is counterintuitive!

let's focus on model complexity



linear model does not fit data as data complexity is higher than the model complexity, **underfitting** occurs $\Rightarrow$ **bias** model

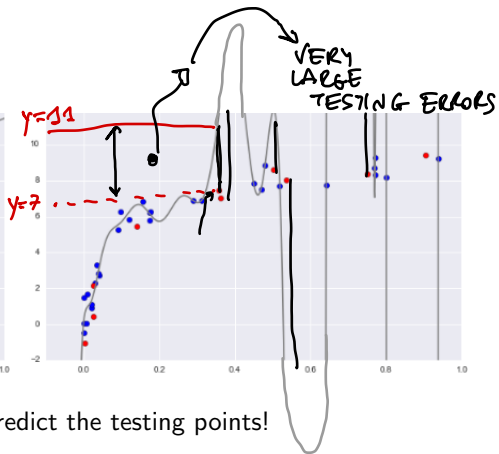
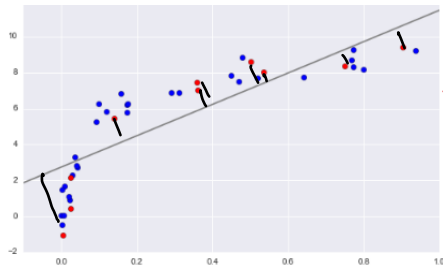


highly nonlinear model has enough complexity to fit data but it does not describe well the process, **overfitting** occurs $\Rightarrow$ **variance** model

DATA COMPLEXITY $\gg$ MODEL COMPLEXITY MODEL COMPLEXITY $\gg$ DATA COMPLEXITY



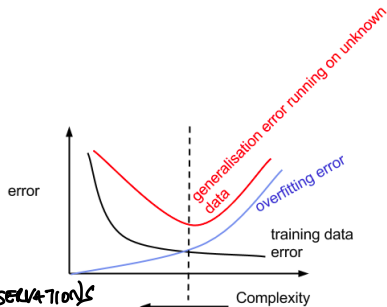
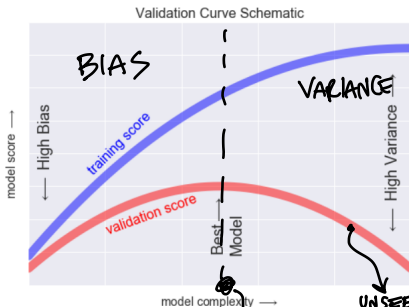
let's focus on model complexity



neither of the two models is able to predict the testing points!

- for highly bias models low accuracy on both training and testing sets
- for highly variance models high accuracy on training set, low on testing set

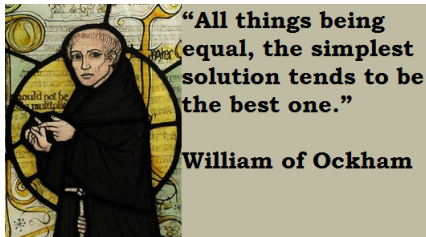
a **trade-off** between bias and variance is required!



the goal is to obtain a model near the maximum of the validation score curve!

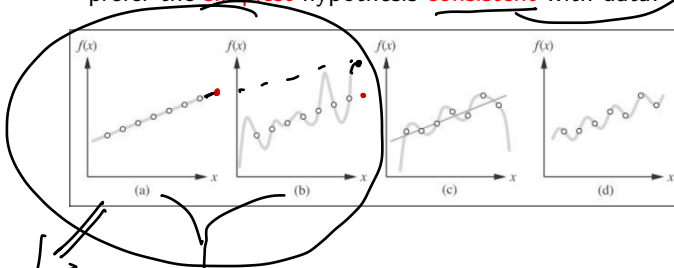
grid-search+cross-validation may help... **GOOD TRADE-OFF BIAS/VARIANCE**

follow the Ockham's razor principle...



prefer the **simplest** hypothesis **consistent** with data!

FITTING
DATA
SUFFICIENTLY
WELL



2) SHOULD BE
PREFERRED

BOTH EQUALLY CONSISTENT WITH DATA